

What CMED Sends AIMS LAB

The complete specification: every field, its name, its type, and what happens to it · AIMScribe v3 · 5 October 2026

Every rule in this document is taken from the program that actually checks the messages when they arrive. If this document and the server ever disagree, the server is right and this is a bug — tell us.

In one sentence

CMED sends **two signals from the doctor's browser** so the recorder knows when a consultation starts and ends, and **two messages from CMED's server** carrying the patient's information and the prescription. **Audio never goes to CMED, and nothing clinical ever comes back.**

1 · The four things CMED builds

	What	From where	When	Carries
API 1	Start recording	the browser	the doctor opens a patient	the five fields
API 2	Patient information	CMED's server	the same moment	demographics, paramedic readings, previous visit
API 3a	Prescription built	the browser	the doctor presses Build Prescription	the five fields and the session id
API 3b	The prescription	CMED's server	the same moment	medicines, diagnoses, investigations

This document covers **API 2 and API 3b** — the two messages from CMED's server that carry patient data. The two browser signals are in a separate integration guide.

	Address
API 2 — patient information	POST https://<aims-lab-server>/api/v2/clinical/patient-information
API 3b — prescription	POST https://<aims-lab-server>/api/v2/clinical/prescription
Header on both	X-CMED-Key: <the key AIMS LAB gives you>
Body	JSON, UTF-8, under 1 MB
Success	HTTP 202 — the message is stored before you get this reply

The key is a server credential. It must never appear in a browser, in page source, or in anything a user can view. Both messages go server to server.

2 · The five fields — the most important rule in this document

Every message about one consultation carries these same five fields, with **exactly the same values, character for character**. They are how we match the prescription to the right recording.

```
{
  "patient_id": "P0012345",
  "doctor_id": "DR0042",
  "hospital_id": "AALO_DHOLPUR",
  "start_time": "2026-10-05T10:14:32+06:00",
  "date": "2026-10-05"
}
```

Field	Type	Rule	Example
patient_id	text	Letters, digits, hyphen, underscore. 1–64 characters. No spaces	P0012345
doctor_id	text	same rule	DR0042

hospital_id	text	same rule. Must be the clinic code AIMS LAB gives you	AALO_DHOLPUR
start_time	text	Date and time with the time-zone offset	2026-10-05T10:14:32+06:00
date	text	The clinic's local date	2026-10-05

Three mistakes that break the match

1. Sending the prescription's time as start_time. API 3b must repeat the **original** start_time from when the patient was opened — not the time the prescription was built. This is the single most common integration mistake.

2. Leaving off the time-zone offset. 2026-10-05T10:14:32 is rejected. +06:00 must be there.

3. Spaces or other characters in an identifier. These become folder names on disk, so only letters, digits, hyphen and underscore are allowed.

3 · API 2 — patient information

Sent the moment the doctor opens the patient, at the same time as the browser signal. It carries everything known **before** the consultation.

```
{
  "patient_id": "P0012345", "doctor_id": "DR0042",
  "hospital_id": "AALO_DHOLPUR",
  "start_time": "2026-10-05T10:14:32+06:00", "date": "2026-10-05",

  "demographics": {
    "name": "Rahima Khatun",
    "sex": "female",
    "age_years": 34,
    "date_of_birth": "1992-03-11",
    "phone": "01700000000",
    "address": "Dholpur, Dhaka"
  },

  "paramedic": {
    "recorded_at": "2026-10-05T10:02:10+06:00",
    "weight_kg": 58.4,
    "height_cm": 157,
    "blood_pressure": "120/80",
    "pulse_bpm": 78,
    "temperature_c": 36.8,
    "spo2_percent": 98,
    "notes": "complains of headache"
  },

  "previous_visit": null,

  "female_details": {},
  "male_details": {}
}
```

3.1 demographics — required

Field name	Type	Accepted	If it is wrong	Goes to
name	text	up to 200 characters. full_name also accepted	shortened	patients.full_name
sex	text	exactly female or male	the whole message is quarantined	patients.sex
age_years	number	0 to 130	stored empty	encounter_demographics
date_of_birth	text	YYYY-MM-DD	stored empty	patients.date_of_birth
phone	text	up to 40 characters	shortened	patients.phone
address	text	up to 500 characters	shortened	patients.address

demographics must be present on every visit, first or fiftieth. **sex is the one field that can fail the whole message** — anything other than the two words is reported back as a problem, so send the word, not a code like M or F.

3.2 paramedic — optional, but send it when you have it

Field name	Type	Accepted	Goes to
recorded_at	text	date and time with offset	paramedic_observations
weight_kg	number	0.5 to 499	weight_kg
height_cm	number	20 to 299	height_cm

blood_pressure	text	"120/80" — upper 30–300, lower 10–250	blood_pressure, and split into systolic and diastolic
pulse_bpm	whole number	10 to 300	pulse_bpm
temperature_c	number	25 to 45	temperature_c
spo2_percent	whole number	0 to 100	spo2_percent
notes	text	up to 2000 characters	notes

A reading outside its range is stored as empty — it never rejects the message. One mistyped temperature must not stop a prescription being saved.

Anything else you put inside paramedic is kept. If you measure something we have not listed — respiration rate, blood sugar, anything — just send it. It is stored in an other field alongside the rest. You do not need our permission and you do not need us to change anything first.

4 · previous_visit — the rule, in full

This field must always be present. Send null when there is nothing to send. Leaving the field out entirely is reported as a problem, because we cannot tell "first visit" from "CMED forgot".

The situation	What to send
The patient's first ever visit	"previous_visit": null
A returning patient, and AIMS LAB does not yet hold the previous visit	the full previous visit, as shown below
A returning patient, and AIMS LAB already holds the previous visit	"previous_visit": null

```
"previous_visit": {
  "date": "2026-06-02",
  "prescription": {
    "items": [
      { "drug": "Metformin 500 mg", "dose": "1 tablet",
        "frequency": "1+0+1", "duration": "30 days",
        "instructions": "after food" }
    ],
    "diagnoses": ["Type 2 diabetes mellitus"],
    "notes": "review in one month"
  }
}
```

Field	Rule
date	Required if you send previous_visit at all. YYYY-MM-DD. The date of that earlier consultation. Without it the whole previous visit is ignored
prescription	Same shape as API 3b, section 5. Only items really matters
diagnoses, notes	May sit inside prescription or beside it — both work

Sending it more often than necessary is completely safe. If you are ever unsure whether we have the previous visit, send it. The database will not duplicate anything: a repeated previous visit updates the one row we already have. **The dangerous direction is the other one** — deciding not to send it when we do not actually have it, which leaves a permanent hole in that patient's history that nobody notices.

Recommendation for the first release: send it every time for every returning patient. It costs about 2 kilobytes. Once we build an endpoint that lets CMED ask us what we already hold, you can switch to sending it only when needed.

5 · API 3b — the prescription

Sent the moment the doctor presses **Build Prescription**. Note that this does **not** stop the recording — the doctor goes on counselling the patient, and that audio is valuable.

```
{
  "patient_id": "P0012345", "doctor_id": "DR0042",
  "hospital_id": "AALO_DHOLPUR",
  "start_time": "2026-10-05T10:14:32+06:00",    <-- the ORIGINAL start time
  "date": "2026-10-05",

  "issued_at": "2026-10-05T10:26:55+06:00",
  "items": [
    { "drug": "Napa 500 mg", "dose": "1 tablet",
      "frequency": "1+1+1", "duration": "5 days",
      "instructions": "after food" },
    { "drug": "Omeprazole 20 mg", "dose": "1 capsule",
      "frequency": "1+0+0", "duration": "14 days",
      "instructions": "before breakfast" }
  ],
  "diagnoses": ["Acute gastritis"],
  "investigations": ["CBC", "HbA1c"],
  "advice": "avoid spicy food",
}
```

```

"follow_up": "2026-11-05",
"notes": "patient counselled about diet"
}

```

Field name	Type	Required ?	Rule	Goes to
issued_at	text	YES	date and time with offset	prescriptions.issued_at
items	list	YES	one entry per medicine. Send [] if none	prescription_items
items[].drug	text	YES	up to 300 characters. Every item needs one	drug
items[].dose	text	no	up to 200	dose
items[].frequency	text	no	up to 200, e.g. 1+1+1	frequency
items[].duration	text	no	up to 200, e.g. 5 days	duration
items[].instructions	text	no	up to 1000	instructions
diagnoses	list of text	YES	Send [] if none	diagnoses
investigations	list of text	YES	Send [] if none	investigations
advice	text	no	up to 2000	prescriptions.advice
follow_up	text	no	YYYY-MM-DD if sent at all	follow_up_date
notes	text	no	up to 2000	prescriptions.notes

An empty list and a missing field are not the same thing. "diagnoses": [] means "the doctor recorded none". Leaving the field out means "CMED did not send it", and that is reported as a problem. Please always send the list.

The order of `items` is kept exactly as you send it, and stored as the line number on the prescription.

6 · Corrections, repeats and retries

Situation	What to do	What happens
The doctor edits the prescription after it was sent	Send API 3b again with the same five fields	We store it as a new version. Both are kept; the newest is the current one
Your network failed, you are not sure it arrived	Send it again, unchanged	Completely safe. You get 202 ALREADY_RECEIVED and one row, not two
You sent something wrong and want to replace it	Send the corrected version with the same five fields	Same as an edit
You sent a field we do not recognise	nothing	It is kept with the rest of the message. Unknown fields are never an error

Every message is stored word for word, exactly as you sent it, before anything is read out of it. So nothing is ever lost because we failed to understand a field. If we later realise we should have been reading something, we can go back through the stored messages and get it.

7 · Every reply, and what to do about it

Code	Means	What CMED should do
202 ACCEPTED	Stored safely	Nothing. Mark it sent
202 ALREADY_RECEIVED	This exact message was already stored	Nothing. Retrying is safe
400 MALFORMED_JSON	The body is not valid JSON	Fix and resend. Do not retry unchanged
400 MISSING_FIELD	One of the five fields is missing or malformed	Fix and resend
400 INVALID_IDENTIFIER	An id has characters outside A–Z a–z 0–9 _ -	Fix and resend
401 INVALID_KEY	The key is missing or wrong	Stop and contact AIMS LAB
413 TOO_LARGE	The body is over 1 MB	Usually a mistake — check for a file or image in the message
422 SCHEMA_INVALID	Stored in quarantine , with a list of which fields were wrong	Not lost. Read the field list in the reply, fix, resend
500 / 503	Our server has a problem	Retry with a growing delay. Never drop the message

Please queue and retry rather than sending once and hoping. If our server is briefly unreachable, the consultation still happened and the recording still exists — the clinical record needs to catch up to it. A message that arrives an hour late is fine. A message that is never sent leaves a recording that is erased after 24 hours.

8 · Rules that are easy to miss

	Rule	Why
1	Never let a message delay the doctor. Send from a background queue	A slow network must never make Build Prescription feel slow
2	start_time in API 3b is the original , from when the patient was opened	It is how the prescription finds its recording
3	Every timestamp carries its offset (+06:00)	A time without a zone is stored as empty
4	previous_visit must be present , even if null	Missing means "CMED forgot"; null means "nothing to send"
5	items, diagnoses and investigations are always lists , empty if none	Same reason

6	sex is the word female or male	Codes like M, F, 1, 2 are rejected
7	hospital_id must be the code AIMS LAB gives you	A recording claiming a clinic the laptop is not enrolled to is refused
8	The key never reaches a browser	It is a server credential
9	Unknown fields are welcome	They are kept. Do not strip data to fit

9 · Where every field ends up

So CMED can see that nothing is thrown away. All of this lives in a database called `aims_clinical`, separate from the one that tracks recordings — which deliberately holds no patient names at all.

You send	We store it in	Holding
the whole message, untouched	<code>intake_records</code>	exactly what you sent, once per message
demographics	<code>patients</code> , <code>encounter_demographics</code>	name, sex, date of birth, phone, address, age at this visit
paramedic	<code>paramedic_observations</code>	each reading in its own column, blood pressure also split into upper and lower, anything unrecognised in other
the visit itself	<code>encounters</code>	one row per consultation, linked to its recording
previous_visit	<code>encounters</code> (marked as a previous visit)	a second row, dated to that earlier day
items	<code>prescription_items</code>	one row per medicine , in the order you sent them
the prescription itself	<code>prescriptions</code>	with a version number, so corrections do not overwrite
diagnoses	<code>diagnoses</code>	one row each
investigations	<code>investigations</code>	one row each
female_details / male_details	their own tables	stored as sent

A copy of all of it is also written as a **JSON file beside the audio recording**, with the same name. So each consultation is one folder containing the audio and its clinical record together, and the database is the index into them.

10 · How CMED can test, today

You do not need to wait for us, and you do not need a real patient.

	What	How
1	Check a message against the rules, with nothing running	<code>python tools/channel_b_test.py --local</code>
2	Send real messages to a test server	<code>python tools/channel_b_test.py --server https://<host> --key <your key></code>
3	See every message and reply side by side	open <code>/protocol-test</code> on the test site

Step 2 runs every case that matters — a good message, the same message twice, a broken one, a changed prescription, a wrong key, an oversized body — and checks each reply against this document. **Please run it and send us the output before the first clinic goes live.**

11 · What each side must hand over

AIMS LAB gives CMED	CMED gives AIMS LAB
The server address	The exact web address of your page (for example <code>https://ehr.aaloclinic.com</code>) — including staging
The Channel B key — shown once	Your own clinic codes , so we can map them to ours
The seven clinic codes	A test environment we can send to

This document and the integration guide	A named contact for integration
---	---------------------------------

The two we need from CMED are blocking. Without your page's exact address the browser cannot talk to the recorder at all. Without your clinic codes, a visit you describe cannot be matched to the clinic it happened in — the record is stored, you get a normal success reply, and the **recording is quietly erased after 24 hours**. That failure is silent from your side, which is why it matters more than it sounds.

12 · One page for the CMED developer

Question	Answer
How many messages do I send?	Two from the browser, two from the server. This document is the two from the server
Where do I send them?	/api/v2/clinical/patient-information and /api/v2/clinical/prescription
How do I authenticate?	Header X-CMED-Key. Server to server only
What ties the messages together?	The five fields, identical in all of them
Biggest trap	start_time in the prescription must be the original , not the time the prescription was built
What is required in API 2?	demographics, and previous_visit present (null is fine)
What is required in API 3b?	issued_at, items, diagnoses, investigations — lists may be empty
What if a reading is out of range?	Stored empty. It never rejects the message
What if I send extra fields?	They are kept. Never an error
Is retrying safe?	Yes. Same body twice gives one row and ALREADY_RECEIVED
What if I get 422?	Nothing is lost — it is quarantined. Read the field list, fix, resend
When do I send previous_visit?	Whenever we might not have it. For the first release, send it every time
Can a message slow the doctor down?	It must not. Queue it and retry in the background
How do I test?	tools/channel_b_test.py, and the /protocol-test page

Prepared by AIMS LAB, United International University. Every rule here is taken from the code that validates these messages on arrival. If anything is unclear, ask before building — it is far cheaper than discovering it in a clinic.